VCGen

SeaBMC Style

Idea

SeaBMC-style VCGen compiles control-dependence into data-dependence.

After preprocessing:

- no block variables are needed
- no separate CFG constraints are emitted
- phi nodes become ordinary `ite` definitions
- cone-of-influence reduction is just data-flow slicing

SeaHorn kept the CFG in the formula; Boogie compiled it into recursion; SeaBMC makes it **disappear**.

Gated SSA

A gamma node is a value-level merge:

$$x := \gamma\big(g, x_t, x_f\big)$$

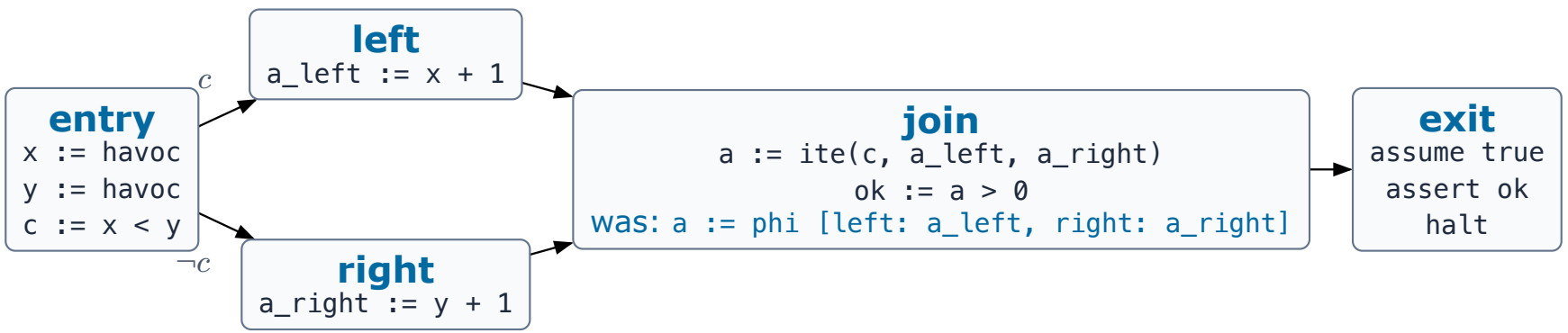
It means:

$$x := \mathrm{ite}\big(g, x_t, x_f\big)$$

Unlike a phi node, `gamma` mentions a Boolean **program expression**, not a predecessor block name.

From Phi To Gamma

The branch condition `c` already decides the incoming region — let the merge switch on **it**:



`a := gamma(c, a_left, a_right)` desugars to `a := ite(c, a_left, a_right)` — an ordinary SSA definition. No CFG predicate needed to explain the merge.

Flat VC

After phi elimination:

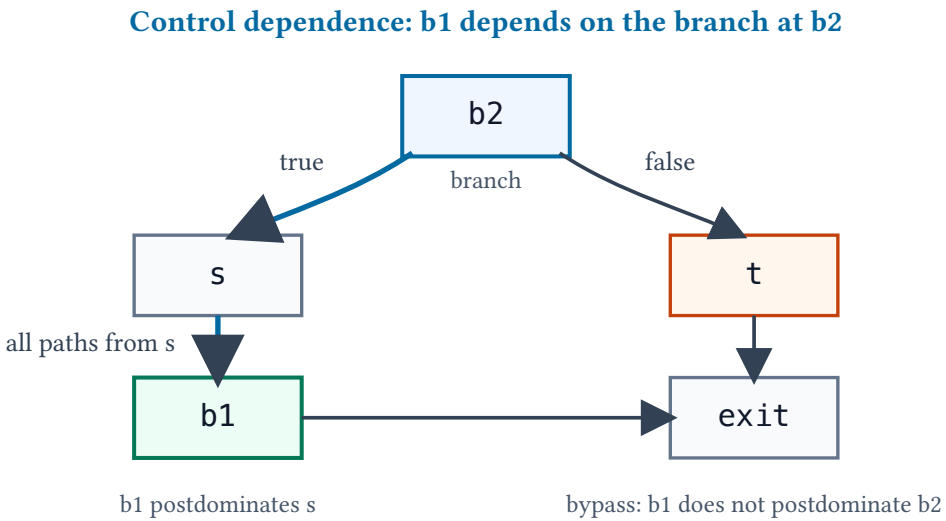
$$\mathrm{VCGen\_SeaBMC}(P) = \mathbf{DEF} \wedge \mathbf{ASSUME} \wedge \neg\, \mathbf{ASSERT}$$

For the diamond:

$$\mathbf{DEF} = c = (x < y) \wedge a_{\mathrm{left}} = x + 1 \wedge a_{\mathrm{right}} = y + 1 \wedge a = \mathrm{ite}\big(c, a_{\mathrm{left}}, a_{\mathrm{right}}\big) \wedge \mathrm{ok} = (a > 0)$$

No block variables, no CFG constraints. The only trace of control flow is the ITE guard in the definition of `a`.

Control Dependence



- d **postdominates** b: every path $b \to$ `exit` passes through d.
- b_1 is **control-dependent** on branch b_2: some successor of b_2 forces a later visit to b_1, while b_1 does not postdominate b_2 itself — the branch genuinely decides.

Gate Construction

For each block b, compute a Boolean expression `gate(b)` over **program** branch conditions:

$$\begin{aligned} \mathrm{gate}(\mathrm{entry}) &= \top \\ \mathrm{gate}(b) &= \bigvee_{c \in \mathrm{ctrl}(b)} \left(\mathrm{gate}(c) \wedge \mathrm{orient}(c, b)\right) \end{aligned}$$

- `ctrl(b)`: branch blocks b is control-dependent on (empty $\longrightarrow \mathrm{gate}(b) = \top$).
- `orient(c,b)`: the branch condition of c, or its negation, depending on which successor reaches b.

In a structured diamond: $\mathrm{gate}(\mathrm{left}) = c$, $\mathrm{gate}(\mathrm{right}) = \neg c$.

The Hidden Blow-Up

`gate` is recursive. Substituted textually into every use, shared control structure is **copied**:

$$\begin{aligned}
\mathrm{gate}(c) &= (p \wedge r) \vee (q \wedge s) \\
\mathrm{gate}(d) &= (q \wedge t) \vee (((p \wedge r) \vee (q \wedge s)) \wedge u) \\
\mathrm{gate}(e) &= ((((p \wedge r) \vee (q \wedge s)) \wedge v) \vee (((q \wedge t) \vee (((p \wedge r) \vee (q \wedge s)) \wedge u)) \wedge w)
\end{aligned}$$

Every level repeats its ancestors; every gamma that uses $\mathrm{gate}(e)$ repeats the whole tree again — **exponential** in the DAG depth.

Materialized Gates

Name each gate as an ordinary Boolean SSA definition, and let gammas refer to the names:

$$G_{\text{entry}} := \top$$
$$G_b := \bigvee_{c \in \text{ctrl}(b)} (G_c \wedge \text{orient}(c, b))$$

For the blow-up example:

$$G_c := (G_a \wedge r) \vee (G_b \wedge s) \qquad G_d := (G_b \wedge t) \vee (G_c \wedge u) \qquad G_e := (G_c \wedge v) \vee (G_d \wedge w)$$

The DAG is represented once — **linear** in gates plus uses. The VC stays CFG-free: gates are just definitions.

Materialized Gates In The Program

Gates are just definitions; the VC stays CFG-free:

```
gate_a := p
gate_b := q
gate_c := (gate_a and r) or (gate_b and s)
gate_d := (gate_b and t) or (gate_c and u)
gate_e := (gate_c and v) or (gate_d and w)

join1:
  x := ite(gate_d, x_d, ite(gate_e, x_e, x_other))

join2:
  y := ite(gate_d, y_d, ite(gate_e, y_e, y_other))
```

`join1` and `join2` share the same control-dependence DAG — it appears once, not once per merge.

Thin GSSA

Materialization stops the copying, but gates can still enumerate every path from `entry`. **Thin GSSA** switches only on the **direct** control-dependence controllers:

$$K_{c,b} = G_c \wedge \mathrm{orient}(c, b)$$

Why the G_c factor? SSA variables are **total** in the formula:

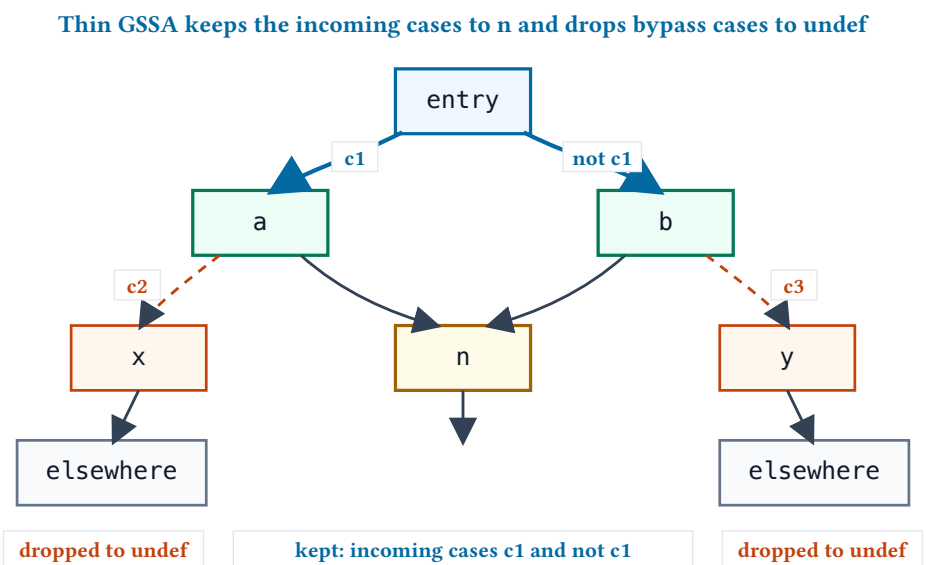
> If controller block c is never reached, its branch condition still has a model value. Switching on the condition alone lets an **unreachable** branch select the merge value.

G_c says "controller c itself is reached" — dead branch conditions cannot fire a case.

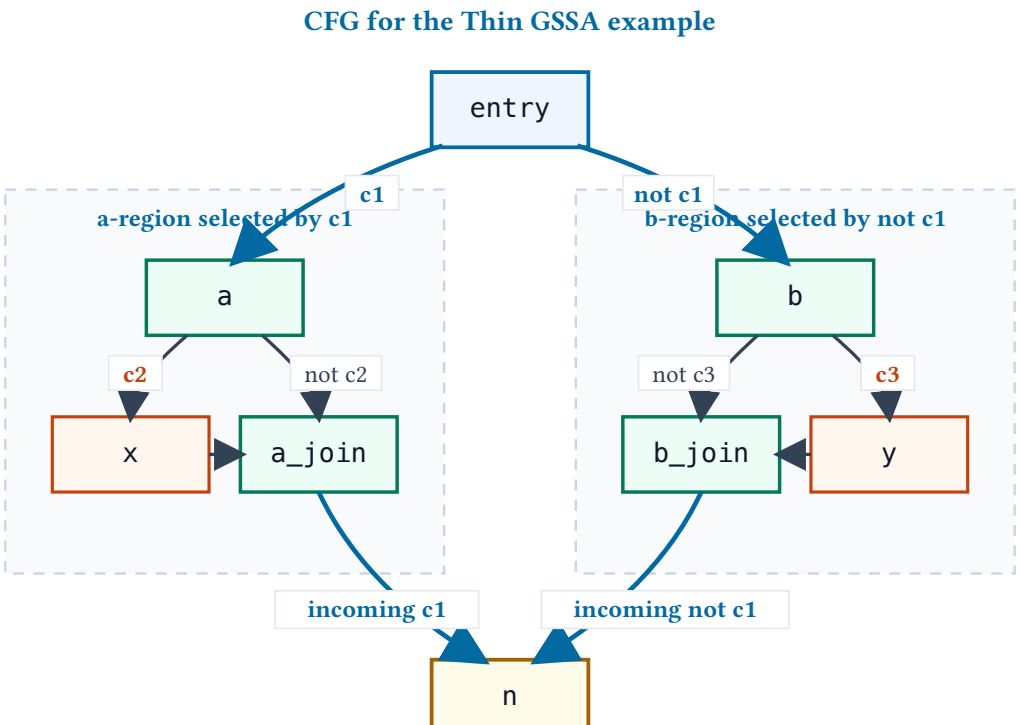
The Fallback: false And undef

If no direct-controller case fires, execution does not reach b:

- For the **reachability gate** G_b, the fallback is `false` — not reached means not reached.
- For a **value** merge in b, the fallback is `undef`: a fresh, unconstrained symbol of the right type. The value is unobservable when b is not reached — any value will do.



Thin GSSA, Derived



The outer branch `c1` selects the region feeding `n`; the local branches `c2`, `c3` only compute values **inside** each region.

Thin GSSA, Derived: Three Steps

Step 1 — regular GSSA: each phi uses the full path condition of its predecessor:

```
a_join:
  v_a := gamma(c1 and c2, v_x, gamma(c1 and not c2, v_a0, undef))

b_join:
  v_b := gamma(not c1 and c3, v_y,
         gamma(not c1 and not c3, v_b0, undef))

n:
  v := gamma(c1, v_a, gamma(not c1, v_b, undef))
```

Every gate repeats the outer condition `c1`.

Thin GSSA, Derived: Three Steps

Step 2 — drop dead fallbacks: the program is SESE, so execution reaching a join arrives from exactly one predecessor — complete two-way choices never select `undef`:

```
a_join:
  v_a := gamma(c2, v_x, gamma(not c2, v_a0, undef))

b_join:
  v_b := gamma(c3, v_y, gamma(not c3, v_b0, undef))

n:
  v := gamma(c1, v_a, v_b)
```

The local gates no longer mention `c1` — the **direct** controller is enough.

Thin GSSA, Derived: Three Steps

Step 3 — thin form, after collapsing the two-way choices and desugaring gamma:

```
a_join:
  v_a := ite(c2, v_x, v_a0)

b_join:
  v_b := ite(c3, v_y, v_b0)

n:
  v := ite(c1, v_a, v_b)
```

The final merge at `n` depends **only** on the region selector `c1`; each local ITE depends only on its own branch. Path conditions never materialize.

Cone Of Influence

Once control is data:

1. Start from the exit `assume` and `assert`.
2. Keep each used variable definition.
3. Add the variables used by that definition.
4. Keep gates only when retained ITEs use them.
5. Drop every unmarked definition.

No graph reachability rule is needed for pruning.

COI Example

```
n:
  v := ite(c1, v_a, v_b)
  junk := expensive(v_x, v_y)
  ok := v > 0

exit:
  assume true
  assert ok
```

`junk` is dropped because it is not used by `ok`, the exit assumption, or a retained gate.

A branch that gates no retained value simply vanishes from the formula.

Three Encodings, One Diamond

	SeaHorn (03)	**Boogie** (04)	**SeaBMC** (05)
control flow	explicit: $\texttt{bb}_i$ variables + path constraints	recursion: backward ok_i equations	compiled away: gates as data
program form	SSA	DSA	SSA → gated SSA
joins	PHI edge implications / ITE	DSA edge definitions	gamma over branch conditions
formula shape	flat conjunction	nested implications	flat conjunction, CFG-free
solver lever	top-level equalities, Boolean propagation on $\texttt{bb}_i$	assertions and assumes for free	substitution and slicing everywhere
watch out for	critical edges, merge exclusivity, unguarded facts	exclusivity for phi/ITE extensions	gate blow-up without materialization

ctac's default encoder (`sea_vc`) is SeaHorn-style — block variables over DSA — with the guarded/unguarded and merge-shape choices exposed as flags.